THỰC HÀNH 1: NHẬP MÔN JAVA & TƯ DUY LẬP TRÌNH

🎯 MỤC TIÊU BÀI HỌC

Sau khi hoàn thành bài thực hành này, bạn sẽ:

1. ✅ **Thiết lập môi trường** Java chuyên nghiệp với VS Code
2. ✅ **Viết chương trình Java** đầu tiên và hiểu cấu trúc cơ bản
3. ✅ **Sử dụng thành thạo** Scanner để nhập dữ liệu từ bàn phím
4. ✅ **Áp dụng cấu trúc điều khiển** (if-else, switch, vòng lặp) vào thực tế
5. ✅ **Làm việc với mảng** và xử lý dữ liệu
6. ✅ **Tư duy logic** như một lập trình viên thực thụ

⏱️ **Thời gian ước tính:** 4-6 giờ

📋 **Kiến thức cần có trước:**

- Đã đọc Chương 1 (phần 0.1 - 0.7): Cấu trúc Java cơ bản, kiểu dữ liệu, toán tử, cấu trúc điều khiển

---

📦 PHẦN 0: CHUẨN BỊ MÔI TRƯỜNG (BẮT BUỘC)

Bước 1: Cài đặt JDK

Windows:

1. Tải JDK 17 hoặc 21 từ: https://adoptium.net/
2. Cài đặt và ghi nhớ đường dẫn (ví dụ: `C:\Program Files\Java\jdk-17`)
3. Thêm vào PATH: `C:\Program Files\Java\jdk-17\bin`

macOS:

```
# Dùng Homebrew
brew install openjdk@17

# Hoặc tải từ Oracle/Adoptium
```

Linux:

```
sudo apt update
sudo apt install openjdk-17-jdk
```

Kiểm tra cài đặt:

```
java -version
javac -version
```

Bước 2: Cài đặt VS Code

1. Tải VS Code: https://code.visualstudio.com/
2. Cài Extension: **Extension Pack for Java** (Microsoft)
3. Khởi động lại VS Code

Bước 3: Tạo Project Đầu Tiên

1. Mở VS Code
2. Nhấn `Ctrl+Shift+P` (Windows) / `Cmd+Shift+P` (Mac)
3. Gõ: `Java: Create Java Project`
4. Chọn: **No Build Tools**
5. Chọn thư mục Desktop
6. Đặt tên: `JavaBootcamp`
7. VS Code mở cửa sổ mới với project

Bước 4: Tạo Package và File Đầu Tiên

1. Trong VS Code, vào thư mục `src`
2. Xóa file `App.java` (nếu có)
3. Chuột phải `src` → **New Java Package**
4. Đặt tên: `com.bootcamp.lab1`
5. Chuột phải package → **New Java Class**
6. Đặt tên: `HelloWorld`

Cấu trúc thư mục:

```
JavaBootcamp/
└── src/
    └── com/
        └── bootcamp/
            └── lab1/
                └── HelloWorld.java
```

🧪 PHẦN 1: HƯỚNG DẪN CHI TIẾT (STEP-BY-STEP)

LAB 1.1: Hello World - Chương Trình Đầu Tiên

Mục tiêu: Viết và chạy chương trình Java đầu tiên

Bước 1: Viết Code

Mở file `HelloWorld.java` và viết:

```java
package com.bootcamp.lab1;

public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Xin chào, đây là chương trình Java đầu tiên
của tôi!");
        System.out.println("Tôi đã sẵn sàng học lập trình!");
    }
}
```

Bước 2: Chạy Chương Trình

Cách 1: Dùng VS Code

- Nhấn `F5` hoặc click nút **Run** (▶) ở góc trên bên phải
- Hoặc chuột phải file → **Run Java**

Cách 2: Dùng Terminal

```bash
cd JavaBootcamp
javac src/com/bootcamp/lab1/HelloWorld.java
java -cp src com.bootcamp.lab1.HelloWorld
```

Bước 3: Kiểm Tra Kết Quả

Bạn sẽ thấy:

```
Xin chào, đây là chương trình Java đầu tiên của tôi!
Tôi đã sẵn sàng học lập trình!
```

🔍 Giải Thích Chi Tiết

1. `package com.bootcamp.lab1;`

- Đây là khai báo package (gói)
- Giống như địa chỉ thư mục để tổ chức code
- Phải khớp với cấu trúc thư mục

2. `public class HelloWorld`

- `public`: Công khai, ai cũng truy cập được
- `class`: Định nghĩa một lớp (sẽ học sâu ở Chương 2)
- `HelloWorld`: Tên class, phải trùng với tên file

3. `public static void main(String[] args)`

- Đây là **điểm khởi đầu** của mọi chương trình Java
- JVM sẽ tìm và chạy method này đầu tiên
- `String[] args`: Nhận tham số từ dòng lệnh (tạm thời chưa dùng)

4. `System.out.println()`

- `System`: Class hệ thống
- `out`: Luồng xuất chuẩn (màn hình console)
- `println`: In và xuống dòng

LAB 1.2: Máy Tính Cá Nhân (Calculator)

Mục tiêu: Hiểu quy trình Input → Process → Output

Yêu Cầu

Tạo chương trình máy tính đơn giản:

- Nhập 2 số
- Chọn phép tính (+, -, *, /)
- Hiển thị kết quả

Hướng Dẫn Từng Bước

Bước 1: Tạo File Mới

Tạo file `Calculator.java` trong package `com.bootcamp.lab1`

Bước 2: Import Scanner

```
package com.bootcamp.lab1;

import java.util.Scanner;  // [1] Import Scanner để nhập liệu

public class Calculator {
    public static void main(String[] args) {
        // Code sẽ viết ở đây
    }
}
```

Giải thích:

- `import`: Nhập khẩu class từ thư viện Java
- `java.util.Scanner`: Class để đọc dữ liệu từ bàn phím
- Giống như bạn cần "mượn" công cụ từ kho chứa

Bước 3: Tạo Scanner Object

```
Scanner scanner = new Scanner(System.in);
```

Giải thích:

- `Scanner`: Kiểu dữ liệu (class)
- `scanner`: Tên biến (có thể đặt tên khác)
- `new Scanner(System.in)`: Tạo object mới, kết nối với bàn phím
- `System.in`: Luồng nhập chuẩn (bàn phím)

Bước 4: Nhập Dữ Liệu (INPUT)

```
System.out.println("=== 🧮 MÁY TÍNH CÁ NHÂN ===");
System.out.print("Nhập số thứ nhất: ");
double so1 = scanner.nextDouble();

System.out.print("Nhập số thứ hai: ");
double so2 = scanner.nextDouble();

System.out.print("Chọn phép tính (+, -, *, /): ");
char phepTinh = scanner.next().charAt(0);
```

Giải thích từng dòng:

1. `System.out.println()`: In và xuống dòng (dùng cho tiêu đề)
2. `System.out.print()`: In nhưng KHÔNG xuống dòng (để user nhập ngay sau câu hỏi)
3. `scanner.nextDouble()`: Đọc số thực từ bàn phím
4. `scanner.next().charAt(0)`:
    - `next()`: Đọc chuỗi
    - `charAt(0)`: Lấy ký tự đầu tiên (vị trí 0)

Bước 5: Xử Lý (PROCESS)

```
double ketQua = 0;
boolean thanhCong = true;  // Cờ đánh dấu thành công

switch (phepTinh) {
    case '+':
        ketQua = so1 + so2;
        break;
    case '-':
        ketQua = so1 - so2;
        break;
    case '*':
        ketQua = so1 * so2;
        break;
    case '/':
        if (so2 == 0) {
            System.out.println("❌ Lỗi: Không thể chia cho 0!");
```

```
            thanhCong = false;
        } else {
            ketQua = so1 / so2;
        }
        break;
    default:
        System.out.println("❌ Lỗi: Phép tính không hợp lệ!");
        thanhCong = false;
}
```

Giải thích:

1. **Switch-Case:**
    - So sánh `phepTinh` với các giá trị
    - `case '+':` Nếu là dấu cộng
    - `break;` Dừng lại, không chạy case tiếp theo
    - `default:` Trường hợp không khớp
2. **Kiểm tra chia cho 0:**
    - Đây là lỗi toán học nghiêm trọng
    - Phải kiểm tra TRƯỚC KHI chia
    - Nếu `so2 == 0` → Báo lỗi, không tính

Bước 6: Hiển Thị Kết Quả (OUTPUT)

```
if (thanhCong) {
    System.out.printf("✅ KẾT QUẢ: %.2f %c %.2f = %.2f\n",
                      so1, phepTinh, so2, ketQua);
}

scanner.close();  // Đóng scanner để giải phóng tài nguyên
```

Giải thích:

1. `System.out.printf()`: In với định dạng
    - `%.2f`: Số thực, làm tròn 2 chữ số thập phân
    - `%c`: Ký tự
    - `\n`: Xuống dòng
2. `scanner.close()`: Quan trọng!
    - Giải phóng tài nguyên
    - Tránh rò rỉ bộ nhớ

Code Hoàn Chỉnh

```java
package com.bootcamp.lab1;

import java.util.Scanner;

public class Calculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("=== 🧮 MÁY TÍNH CÁ NHÂN ===");

        // INPUT
        System.out.print("Nhập số thứ nhất: ");
        double so1 = scanner.nextDouble();

        System.out.print("Nhập số thứ hai: ");
        double so2 = scanner.nextDouble();

        System.out.print("Chọn phép tính (+, -, *, /): ");
        char phepTinh = scanner.next().charAt(0);

        // PROCESS
        double ketQua = 0;
        boolean thanhCong = true;

        switch (phepTinh) {
            case '+':
                ketQua = so1 + so2;
                break;
            case '-':
                ketQua = so1 - so2;
                break;
            case '*':
                ketQua = so1 * so2;
                break;
            case '/':
                if (so2 == 0) {
                    System.out.println("❌ Lỗi: Không thể chia cho 0!");
                    thanhCong = false;
                } else {
                    ketQua = so1 / so2;
                }
                break;
            default:
                System.out.println("❌ Lỗi: Phép tính không hợp lệ!");
                thanhCong = false;
        }

        // OUTPUT
        if (thanhCong) {
            System.out.printf("✅ KẾT QUẢ: %.2f %c %.2f = %.2f\n",
                              so1, phepTinh, so2, ketQua);
        }
```

```
        scanner.close();
    }
}
```

⚠ Lỗi Thường Gặp

1. Lỗi "Trôi lệnh" (Scanner Buffer Issue)

Vấn đề:

```
System.out.print("Nhập tuổi: ");
int tuoi = scanner.nextInt();
System.out.print("Nhập tên: ");
String ten = scanner.nextLine();  // ❌ Bị bỏ qua!
```

Nguyên nhân:

- `nextInt()` chỉ đọc số, để lại ký tự Enter (`\n`) trong buffer
- `nextLine()` đọc trúng Enter đó và tưởng user đã nhập xong

Giải pháp:

```
System.out.print("Nhập tuổi: ");
int tuoi = scanner.nextInt();
scanner.nextLine();  // ✅ Nuốt ký tự Enter thừa

System.out.print("Nhập tên: ");
String ten = scanner.nextLine();  // ✅ Hoạt động đúng
```

2. Quên đóng Scanner

Vấn đề:

```
Scanner scanner = new Scanner(System.in);
// ... code ...
// Quên scanner.close()
```

Hậu quả: Rò rỉ tài nguyên, warning từ IDE

Giải pháp: Luôn gọi `scanner.close()` ở cuối

LAB 1.3: Quản Lý Điểm Số (Grade Analyzer)

Mục tiêu: Làm chủ mảng và vòng lặp

Yêu Cầu

Tạo chương trình:

- Nhập số lượng sinh viên
- Nhập điểm từng sinh viên (0-10)
- Tính điểm trung bình, cao nhất, thấp nhất
- Xếp loại: ĐẬU (>=5) hoặc RỚT (<5)

Hướng Dẫn Từng Bước

Bước 1: Khai Báo và Nhập Số Lượng

```
package com.bootcamp.lab1;

import java.util.Scanner;

public class GradeAnalyzer {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Lớp có bao nhiêu sinh viên? ");
        int soLuong = sc.nextInt();
```

Bước 2: Tạo Mảng

```
        // Tạo mảng chứa điểm
        double[] diemSo = new double[soLuong];
```

Giải thích:

- `double[]`: Kiểu mảng số thực
- `diemSo`: Tên mảng
- `new double[soLuong]`: Tạo mảng với `soLuong` phần tử
- Giá trị mặc định: `0.0` cho tất cả phần tử

Bước 3: Nhập Điểm với Validation

```
        System.out.println("--- NHẬP ĐIỂM ---");
        for (int i = 0; i < soLuong; i++) {
            do {
                System.out.printf("Nhập điểm SV thứ %d (0-10): ", (i +
1));
                diemSo[i] = sc.nextDouble();

                if (diemSo[i] < 0 || diemSo[i] > 10) {
                    System.out.println("⚠ Điểm phải từ 0 đến 10. Mời nhập
```

```
lại!");
                }
            } while (diemSo[i] < 0 || diemSo[i] > 10);
        }
```

Giải thích:

1. **Vòng lặp for:**
   - `int i = 0`: Bắt đầu từ 0 (index mảng bắt đầu từ 0)
   - `i < soLuong`: Chạy đến khi `i` bằng `soLuong - 1`
   - `i++`: Tăng `i` lên 1 sau mỗi lần lặp
2. **Vòng lặp do-while:**
   - `do { ... } while (điều kiện)`
   - Chạy ít nhất 1 lần
   - Lặp lại nếu điều kiện đúng
   - Phù hợp cho validation (nhập lại nếu sai)
3. **Validation:**
   - Kiểm tra điểm trong khoảng [0, 10]
   - Nếu sai → Báo lỗi và nhập lại

Bước 4: Tính Toán và Phân Tích

```
        // Khởi tạo biến
        double tong = 0;
        double diemCaoNhat = diemSo[0];  // Giả sử người đầu tiên cao nhất
        double diemThapNhat = diemSo[0];  // Giả sử người đầu tiên thấp
nhất

        System.out.println("\n--- BẢNG XẾP HẠNG ---");
        for (int i = 0; i < soLuong; i++) {
            // Cộng dồn để tính tổng
            tong += diemSo[i];

            // Tìm điểm cao nhất
            if (diemSo[i] > diemCaoNhat) {
                diemCaoNhat = diemSo[i];
            }

            // Tìm điểm thấp nhất
            if (diemSo[i] < diemThapNhat) {
                diemThapNhat = diemSo[i];
            }

            // Xếp loại
            String xepLoai = (diemSo[i] >= 5) ? "ĐẬU" : "RỚT";
            System.out.printf("SV %d: %.1f -> %s\n", (i+1), diemSo[i],
```

```
xepLoai);
        }
```

Giải thích:

1. **Thuật toán tìm Max/Min:**
   - Giả sử phần tử đầu tiên là Max/Min
   - Duyệt qua từng phần tử
   - Nếu tìm thấy lớn hơn → Cập nhật Max
   - Nếu tìm thấy nhỏ hơn → Cập nhật Min
2. **Toán tử điều kiện (Ternary):**
   - `(điều kiện) ? giá trị đúng : giá trị sai`
   - Ngắn gọn hơn if-else đơn giản

Bước 5: Hiển Thị Kết Quả

```
        System.out.println("----------------------");
        System.out.printf("⭐ Điểm cao nhất: %.1f\n", diemCaoNhat);
        System.out.printf("🔻 Điểm thấp nhất: %.1f\n", diemThapNhat);
        System.out.printf("📊 Điểm trung bình: %.2f\n", (tong / soLuong));

        sc.close();
    }
}
```

Code Hoàn Chỉnh

```
package com.bootcamp.lab1;

import java.util.Scanner;

public class GradeAnalyzer {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Lớp có bao nhiêu sinh viên? ");
        int soLuong = sc.nextInt();

        double[] diemSo = new double[soLuong];

        System.out.println("--- NHẬP ĐIỂM ---");
        for (int i = 0; i < soLuong; i++) {
            do {
                System.out.printf("Nhập điểm SV thứ %d (0-10): ", (i +
1));
                diemSo[i] = sc.nextDouble();
```

```
                if (diemSo[i] < 0 || diemSo[i] > 10) {
                    System.out.println("⚠ Điểm phải từ 0 đến 10. Mời nhập
lại!");
                }
            } while (diemSo[i] < 0 || diemSo[i] > 10);
        }

        double tong = 0;
        double diemCaoNhat = diemSo[0];
        double diemThapNhat = diemSo[0];

        System.out.println("\n--- BẢNG XẾP HẠNG ---");
        for (int i = 0; i < soLuong; i++) {
            tong += diemSo[i];

            if (diemSo[i] > diemCaoNhat) {
                diemCaoNhat = diemSo[i];
            }
            if (diemSo[i] < diemThapNhat) {
                diemThapNhat = diemSo[i];
            }

            String xepLoai = (diemSo[i] >= 5) ? "ĐẬU" : "RỚT";
            System.out.printf("SV %d: %.1f -> %s\n", (i+1), diemSo[i],
xepLoai);
        }

        System.out.println("----------------------");
        System.out.printf("⭐ Điểm cao nhất: %.1f\n", diemCaoNhat);
        System.out.printf("🔻 Điểm thấp nhất: %.1f\n", diemThapNhat);
        System.out.printf("📊 Điểm trung bình: %.2f\n", (tong / soLuong));

        sc.close();
    }
}
```

📝 PHẦN 2: BÀI TẬP CÓ HƯỚNG DẪN

Bài 2.1: Kiểm Tra Số Nguyên Tố

Đề bài: Nhập số nguyên `n`. Kiểm tra `n` có phải số nguyên tố không.

Số nguyên tố: Chỉ chia hết cho 1 và chính nó (ví dụ: 2, 3, 5, 7, 11, ...)

Hướng Dẫn

Bước 1: Phân Tích Logic

1. Nếu `n < 2` → Không phải số nguyên tố
2. Nếu `n == 2` → Là số nguyên tố

3. Nếu n chẵn và n > 2 → Không phải số nguyên tố
4. Kiểm tra từ 3 đến √n (căn bậc 2 của n):
    - Nếu n chia hết cho bất kỳ số nào → Không phải số nguyên tố
    - Nếu không chia hết cho số nào → Là số nguyên tố

Bước 2: Viết Code

```java
package com.bootcamp.lab1;

import java.util.Scanner;

public class KiemTraSoNguyenTo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Nhập số nguyên n: ");
        int n = sc.nextInt();

        boolean laSoNguyenTo = true;

        // Trường hợp đặc biệt
        if (n < 2) {
            laSoNguyenTo = false;
        } else if (n == 2) {
            laSoNguyenTo = true;
        } else if (n % 2 == 0) {
            // Số chẵn > 2 không phải số nguyên tố
            laSoNguyenTo = false;
        } else {
            // Kiểm tra từ 3 đến căn bậc 2 của n (chỉ kiểm tra số lẻ)
            for (int i = 3; i <= Math.sqrt(n); i += 2) {
                if (n % i == 0) {
                    laSoNguyenTo = false;
                    break;  // Tìm thấy ước số → Dừng ngay
                }
            }
        }

        // Hiển thị kết quả
        if (laSoNguyenTo) {
            System.out.println(n + " là số nguyên tố");
        } else {
            System.out.println(n + " không phải số nguyên tố");
        }

        sc.close();
    }
}
```

Giải thích:

1. `Math.sqrt(n)`:
   - Tính căn bậc 2 của `n`
   - Tối ưu: Chỉ cần kiểm tra đến `√n` (vì nếu có ước > `√n` thì cũng có ước < `√n`)
2. `i += 2`:
   - Tăng `i` lên 2 mỗi lần (chỉ kiểm tra số lẻ)
   - Tối ưu: Đã loại số chẵn ở trên
3. `break`:
   - Thoát khỏi vòng lặp ngay khi tìm thấy ước số
   - Tối ưu: Không cần kiểm tra tiếp

Bài 2.2: Bảng Cửu Chương

Đề bài: In bảng cửu chương từ 2 đến 9.

Hướng Dẫn

Phân tích:

- Vòng lặp ngoài: Từ 2 đến 9 (số nhân)
- Vòng lặp trong: Từ 1 đến 10 (số bị nhân)

Code:

```java
package com.bootcamp.lab1;

public class BangCuuChuong {
    public static void main(String[] args) {
        System.out.println("=== BẢNG CỬU CHƯƠNG ===\n");

        // Vòng lặp ngoài: Từ 2 đến 9
        for (int i = 2; i <= 9; i++) {
            System.out.println("Bảng nhân " + i + ":");

            // Vòng lặp trong: Từ 1 đến 10
            for (int j = 1; j <= 10; j++) {
                System.out.printf("%d x %d = %d\n", i, j, i * j);
            }

            System.out.println();  // Xuống dòng trống giữa các bảng
        }
    }
}
```

Kết quả:

```
=== BẢNG CỬU CHƯƠNG ===

Bảng nhân 2:
2 x 1 = 2
2 x 2 = 4
...
```

Bài 2.3: Đếm Số Chẵn/Lẻ trong Mảng

Đề bài: Nhập mảng n số nguyên. Đếm số chẵn và số lẻ.

Hướng Dẫn

Code:

```java
package com.bootcamp.lab1;

import java.util.Scanner;

public class DemChanLe {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Nhập số lượng phần tử: ");
        int n = sc.nextInt();

        int[] mang = new int[n];

        // Nhập mảng
        System.out.println("Nhập các phần tử:");
        for (int i = 0; i < n; i++) {
            System.out.print("Phần tử thứ " + (i + 1) + ": ");
            mang[i] = sc.nextInt();
        }

        // Đếm chẵn/lẻ
        int demChan = 0;
        int demLe = 0;

        for (int i = 0; i < n; i++) {
            if (mang[i] % 2 == 0) {
                demChan++;
            } else {
                demLe++;
            }
        }

        // Hiển thị kết quả
        System.out.println("\n--- KẾT QUẢ ---");
```

```
        System.out.println("Số chẵn: " + demChan);
        System.out.println("Số lẻ: " + demLe);

        sc.close();
    }
}
```

Giải thích:

- `mang[i] % 2 == 0`: Kiểm tra số chẵn (chia 2 dư 0)
- `demChan++`: Tăng biến đếm lên 1

🎯 PHẦN 3: BÀI TẬP TỰ LUYỆN (KHÔNG CÓ HƯỚNG DẪN)

Bài 3.1: Tính Giai Thừa

Đề bài: Nhập số nguyên `n`. Tính `n!` (giai thừa).

Gợi ý:

- `n! = 1 × 2 × 3 × ... × n`
- Ví dụ: `5! = 1 × 2 × 3 × 4 × 5 = 120`
- Lưu ý: `0! = 1`

Yêu cầu:

- Kiểm tra `n >= 0`
- Hiển thị kết quả

Bài 3.2: Tìm Số Lớn Nhất và Nhỏ Nhất

Đề bài: Nhập mảng `n` số nguyên. Tìm số lớn nhất và nhỏ nhất.

Yêu cầu:

- Không dùng `Arrays.sort()`
- Tự viết thuật toán tìm kiếm

Bài 3.3: Đảo Ngược Mảng

Đề bài: Nhập mảng `n` số nguyên. Đảo ngược thứ tự mảng.

Ví dụ:

- Input: `[1, 2, 3, 4, 5]`
- Output: `[5, 4, 3, 2, 1]`

Gợi ý:

- Hoán đổi phần tử đầu với cuối

- Dùng biến tạm để swap

Bài 3.4: Kiểm Tra Mảng Đối Xứng

Đề bài: Nhập mảng n số nguyên. Kiểm tra mảng có đối xứng không.

Ví dụ:

- `[1, 2, 3, 2, 1]` → Đối xứng
- `[1, 2, 3, 4, 5]` → Không đối xứng

Gợi ý:

- So sánh phần tử đầu với cuối
- So sánh phần tử thứ 2 với phần tử thứ 2 từ cuối
- ...

Bài 3.5: Tính Tổng Các Số Chẵn

Đề bài: Nhập mảng n số nguyên. Tính tổng các số chẵn.

Yêu cầu:

- Chỉ tính tổng số chẵn
- Hiển thị cả số lượng và tổng

Bài 3.6: Tìm Số Nguyên Tố trong Mảng

Đề bài: Nhập mảng n số nguyên dương. Tìm và hiển thị tất cả số nguyên tố trong mảng.

Gợi ý:

- Số nguyên tố chỉ chia hết cho 1 và chính nó
- Kiểm tra từ 2 đến căn bậc 2 của số đó
- Tạo phương thức `isPrime(int n)` để kiểm tra

Yêu cầu:

- Hiển thị vị trí và giá trị của các số nguyên tố
- Nếu không có số nguyên tố nào, thông báo phù hợp

Bài 3.7: Tính Trung Bình Cộng Các Số Dương

Đề bài: Nhập mảng n số nguyên (có thể âm, dương, hoặc 0). Tính trung bình cộng chỉ các số dương.

Gợi ý:

- Chỉ cộng các số > 0
- Đếm số lượng số dương

- Chia tổng cho số lượng

Yêu cầu:

- Xử lý trường hợp không có số dương nào
- Làm tròn kết quả đến 2 chữ số thập phân

Bài 3.8: Tìm Phần Tử Xuất Hiện Nhiều Nhất

Đề bài: Nhập mảng n số nguyên. Tìm phần tử xuất hiện nhiều lần nhất và số lần xuất hiện.

Gợi ý:

- Dùng 2 vòng lặp lồng nhau để đếm
- Vòng ngoài: duyệt từng phần tử
- Vòng trong: đếm số lần phần tử đó xuất hiện
- Lưu phần tử có số lần xuất hiện cao nhất

Yêu cầu:

- Nếu có nhiều phần tử cùng số lần xuất hiện, chọn phần tử đầu tiên
- Hiển thị phần tử và số lần xuất hiện

Bài 3.9: Sắp Xếp Mảng Tăng Dần (Selection Sort)

Đề bài: Nhập mảng n số nguyên. Sắp xếp mảng tăng dần bằng thuật toán Selection Sort (KHÔNG dùng `Arrays.sort()`).

Gợi ý:

- Selection Sort: Tìm phần tử nhỏ nhất, đưa lên đầu
- Vòng ngoài: chọn vị trí cần điền
- Vòng trong: tìm phần tử nhỏ nhất từ vị trí hiện tại đến cuối
- Hoán đổi phần tử nhỏ nhất với phần tử ở vị trí đang xét

Yêu cầu:

- Hiển thị mảng trước và sau khi sắp xếp
- Hiển thị từng bước sắp xếp (optional)

Bài 3.10: Tách Mảng Thành Mảng Chẵn và Mảng Lẻ

Đề bài: Nhập mảng n số nguyên. Tách thành 2 mảng: một mảng chứa số chẵn, một mảng chứa số lẻ.

Gợi ý:

- Đếm trước số chẵn và số lẻ để tạo mảng đúng kích thước
- Hoặc dùng mảng tạm lớn, sau đó copy sang mảng đúng kích thước
- Duyệt mảng gốc, phân loại vào mảng chẵn hoặc lẻ

Yêu cầu:

- Hiển thị mảng gốc, mảng chẵn, mảng lẻ
- Xử lý trường hợp toàn chẵn hoặc toàn lẻ

🏆 PHẦN 4: THỬ THÁCH NÂNG CAO

Challenge 1: Trò Chơi Đoán Số (Guess The Number)

Luật chơi:

1. Máy tính sinh số ngẫu nhiên từ 1 đến 100
2. Người chơi đoán số
3. Máy gợi ý "Lớn hơn" hoặc "Nhỏ hơn"
4. Chỉ cho phép đoán tối đa 7 lần
5. Quá 7 lần → Game Over

Gợi ý:

- Dùng `Math.random()` để sinh số ngẫu nhiên
- `Math.random()` trả về số từ 0.0 đến 1.0
- Để có số từ 1-100: `(int)(Math.random() * 100) + 1`

Yêu cầu:

- Hiển thị số lần đoán còn lại
- Khi thắng: Hiển thị số lần đoán đã dùng
- Khi thua: Hiển thị số đúng

Challenge 2: Máy ATM Đơn Giản

Chức năng:

1. **Login:** Mã PIN cố định là `1234`
   - Sai 3 lần → Khóa thẻ
2. **Menu:**
   - `1. Xem số dư`
   - `2. Rút tiền`
   - `3. Nạp tiền`
   - `4. Thoát`
3. **Rút tiền:**
   - Phải là bội số của 50,000
   - Không quá số dư hiện có
4. **Nạp tiền:**
   - Phải > 0

Yêu cầu:

- Sử dụng vòng lặp `while` cho menu

- Validation đầy đủ
- Giao diện thân thiện

Challenge 3: Sắp Xếp Mảng (Bubble Sort)

Đề bài: Nhập mảng số nguyên. Tự viết thuật toán sắp xếp từ bé đến lớn (KHÔNG dùng `Arrays.sort()`).

Thuật toán Bubble Sort:

1. So sánh 2 phần tử cạnh nhau
2. Nếu phần tử trước > phần tử sau → Đổi chỗ
3. Lặp lại cho đến khi mảng được sắp xếp

Gợi ý:

- Dùng 2 vòng lặp lồng nhau
- Vòng ngoài: Số lần duyệt
- Vòng trong: So sánh và đổi chỗ

Ví dụ:

- Input: `[5, 2, 8, 1, 9]`
- Output: `[1, 2, 5, 8, 9]`

📚 BẢNG GHI NHỚ (CHEAT SHEET)

Scanner - Nhập Liệu

Code	Ý nghĩa	Ví dụ
`Scanner sc = new Scanner(System.in);`	Tạo Scanner	Chuẩn bị nhập từ bàn phím
`sc.nextInt()`	Nhập số nguyên	`int age = sc.nextInt();`
`sc.nextDouble()`	Nhập số thực	`double price = sc.nextDouble();`
`sc.nextLine()`	Nhập cả dòng chữ	`String name = sc.nextLine();`
`sc.next()`	Nhập từ (đến khoảng trắng)	`String word = sc.next();`
`sc.next().charAt(0)`	Nhập ký tự	`char op = sc.next().charAt(0);`
`sc.close()`	Đóng Scanner	Giải phóng tài nguyên

Mảng (Array)

Code	Ý nghĩa	Ví dụ
`int[] arr = new int[10];`	Tạo mảng 10 phần tử	Mảng số nguyên
`arr.length`	Độ dài mảng	`for (int i = 0; i < arr.length; i++)`
`arr[0]`	Phần tử đầu tiên	Index bắt đầu từ 0
`arr[arr.length - 1]`	Phần tử cuối cùng	

Vòng Lặp

Loại	Cú pháp	Khi nào dùng
for	`for (int i = 0; i < n; i++)`	Biết trước số lần lặp
while	`while (điều kiện)`	Không biết trước số lần
do-while	`do { ... } while (điều kiện)`	Cần chạy ít nhất 1 lần

Toán Tử

Toán tử	Ý nghĩa	Ví dụ
`%`	Chia lấy dư	`5 % 2 = 1` (5 chia 2 dư 1)
`++`	Tăng lên 1	`i++` tương đương `i = i + 1`
`--`	Giảm xuống 1	`i--` tương đương `i = i - 1`
`+=`	Cộng và gán	`tong += 5` tương đương `tong = tong + 5`

✅ CHECKLIST HOÀN THÀNH

Sau khi làm xong bài thực hành này, hãy đánh dấu:

- ☐ Đã cài đặt JDK và VS Code thành công
- ☐ Đã tạo project Java đầu tiên
- ☐ Đã chạy được chương trình HelloWorld
- ☐ Đã hoàn thành LAB 1.2 (Calculator)
- ☐ Đã hoàn thành LAB 1.3 (Grade Analyzer)
- ☐ Đã làm ít nhất 3 bài tập có hướng dẫn
- ☐ Đã làm ít nhất 2 bài tập tự luyện
- ☐ Đã thử ít nhất 1 thử thách nâng cao

🎓 TỔNG KẾT

Sau bài thực hành này, bạn đã:

1. ✅ **Thiết lập môi trường** Java chuyên nghiệp

2. ✅ **Hiểu cấu trúc** chương trình Java cơ bản
3. ✅ **Sử dụng Scanner** để nhập liệu
4. ✅ **Áp dụng cấu trúc điều khiển** (if-else, switch, vòng lặp)
5. ✅ **Làm việc với mảng** và xử lý dữ liệu
6. ✅ **Tư duy logic** để giải quyết bài toán

Bước tiếp theo: Chuyển sang Thực hành 2: Tư duy Hướng đối tượng (OOP Core)

Chúc bạn học tốt! 🚀